BÁO CÁO TUẦN 2: COLLECTIONS VÀ DESIGN PATTERNS TRONG JAVA

I. Java Collections Framework

2.1. Tổng quan về Java Collections Framework

Java Collections Framework là tập hợp các interface, class và thuật toán được Java cung cấp để lưu trữ, quản lý và thao tác với nhóm đối tượng.

Các interface thường gặp:

Interface	Chức năng
List	Lưu trữ phần tử theo thứ tự, cho phép trùng lặp
Set	Không cho phép các phần tử trùng nhau theo quy tắc của tập hợp
Queue	Quản lý phần tử theo quy tắc hàng đợi
Map	Lưu trữ dữ liệu theo cặp key-value

Lưu ý: Map thuộc Collections Framework nhưng không kế thừa interface Collection.

Một số class triển khai phổ biến:

- ArrayList: triển khai List, sử dụng mảng động.
- HashSet: triển khai Set, dựa trên cơ chế băm.
- HashMap: triển khai Map, lưu trữ các cặp key-value.
- ConcurrentHashMap: hỗ trợ thao tác trên Map trong môi trường đa luồng.
- LinkedList: triển khai List và Deque, sử dụng các node liên kết.

2.2. ArrayList

2.2.1. Khái niệm

ArrayList là một cấu trúc dữ liệu triển khai interface List, cho phép lưu trữ nhiều phần tử theo thứ tự và thay đổi kích thước trong quá trình chạy.

Ví dụ:

```
import java.util.ArrayList;

import java.util.List;


public class Main {

    public static void main(String[] args) {

        List<String> names = new ArrayList<>();
```

```
        names.add("Duy");
        names.add("An");
        names.add("Minh");
        names.add("Duy");

        System.out.println(names);
        System.out.println(names.get(1));
    }
}
```

Kết quả:

[Duy, An, Minh, Duy]

An

ArrayList cho phép phần tử trùng lặp và duy trì thứ tự thêm vào. Nó cũng cho phép lưu null.

2.2.2. Cấu trúc bên trong của ArrayList

ArrayList được triển khai dựa trên một mảng nội bộ có khả năng mở rộng, thường được gọi là *dynamic array*.

Về bản chất, các phần tử được lưu trong một mảng các tham chiếu đến đối tượng. Khi số lượng phần tử tăng vượt sức chứa hiện tại, ArrayList sẽ tạo mảng mới có sức chứa lớn hơn rồi sao chép các tham chiếu sang mảng mới.

Ví dụ, giả sử mảng có sức chứa 4 phần tử:

Mảng nội bộ:

```
Index:   0      1      2      3
       +------+------+------+------+
       | Duy  | An   | Minh | null |
       +------+------+------+------+
```

Số phần tử thực tế: 3

Sức chứa: 4

Khi thêm phần tử thứ năm, mảng có thể phải mở rộng:

Trước khi mở rộng:

[Duy, An, Minh, Lan]

Thêm "Hoa" -> vượt sức chứa

Sau khi mở rộng:

[Duy, An, Minh, Lan, Hoa, null, ...]

Mảng mới được cấp phát và các tham chiếu được sao chép từ mảng cũ. Đối tượng String không bị nhân bản; chỉ các tham chiếu được chuyển sang mảng mới.

Trong các phiên bản OpenJDK hiện đại, khi ArrayList cần mở rộng thông thường, sức chứa mới thường tăng khoảng 50% so với sức chứa cũ. Đây là chi tiết triển khai, không phải cam kết cố định của mọi phiên bản Java.

2.2.3. Cơ chế hoạt động của các phương thức

Phương thức add(element)

- Nếu mảng còn chỗ, phần tử được đặt vào vị trí tiếp theo.
- Nếu mảng hết chỗ, ArrayList mở rộng mảng rồi thêm phần tử.

Phương thức get(index)

- Truy cập phần tử trực tiếp thông qua chỉ số mảng.
- Không cần duyệt tuần tự qua các phần tử trước đó.

Phương thức remove(index)

- Xóa phần tử ở vị trí được chỉ định.
- Các phần tử phía sau được dịch sang trái để lấp khoảng trống.

Ví dụ:

Ban đầu: [A, B, C, D]

remove(1): [A, C, D]

Phần tử B bị xóa, còn C và D được dịch sang trái.

2.2.4. Độ phức tạp thời gian

Thao tác	Độ phức tạp thông thường

get(index)	O(1)
set(index, value)	O(1)
add(value) vào cuối	O(1) trung bình khấu hao
add(index, value)	O(n)
remove(index)	O(n) trong trường hợp tổng quát
contains(value)	O(n)

Giải thích về O(1) trung bình khấu hao: Phần lớn thao tác thêm vào cuối rất nhanh. Tuy nhiên, thỉnh thoảng ArrayList phải tạo mảng mới và sao chép các phần tử, khiến một lần thêm có thể mất O(n). Khi xét một chuỗi nhiều thao tác thêm, chi phí trung bình trên mỗi lần thêm vẫn là O(1).

2.2.5. Ưu điểm và hạn chế

Ưu điểm:

- Truy cập phần tử theo index nhanh.
- Dễ sử dụng, phù hợp với nhiều tình huống.
- Thêm phần tử cuối hiệu quả trong đa số trường hợp.
- Duy trì thứ tự phần tử.

Hạn chế:

- Chèn hoặc xóa ở đầu hay giữa danh sách có thể tốn chi phí dịch chuyển.
- Tăng kích thước có thể cần cấp phát và sao chép mảng.
- Không an toàn khi nhiều luồng đồng thời sửa đổi mà không có cơ chế đồng bộ thích hợp.

2.2.6. Khi nào nên sử dụng ArrayList?

Nên sử dụng ArrayList khi:

- Cần lưu một danh sách phần tử có thứ tự.
- Thường xuyên đọc dữ liệu bằng index.
- Chủ yếu thêm phần tử vào cuối.
- Không yêu cầu tự động đồng bộ giữa nhiều luồng.

Ví dụ thực tế: lưu danh sách sản phẩm trong giỏ hàng, danh sách nhân viên hoặc kết quả tìm kiếm.

2.3. HashMap

2.3.1. Khái niệm

HashMap là một cấu trúc dữ liệu triển khai interface Map, dùng để lưu trữ dữ liệu theo cặp key-value.

Mỗi key tương ứng với một value. Các key phải duy nhất theo quy tắc so sánh của Map, trong khi các value có thể trùng nhau.

Ví dụ:

```
import java.util.HashMap;

import java.util.Map;


public class Main {

    public static void main(String[] args) {

        Map<Integer, String> employees = new HashMap<>();


        employees.put(101, "Duy");

        employees.put(102, "An");

        employees.put(103, "Minh");


        System.out.println(employees.get(102));


        employees.put(102, "Hoang");


        System.out.println(employees.get(102));

    }

}
```

Kết quả:

```
An

Hoang
```

Khi thêm một entry có key đã tồn tại, HashMap thay thế value cũ bằng value mới. Nó không tạo thêm một key trùng lặp.

2.3.2. Cấu trúc bên trong của HashMap

HashMap sử dụng một mảng các bucket để lưu trữ các entry. Mỗi entry chứa thông tin về hash, key, value và liên kết đến entry tiếp theo nếu các entry nằm trong cùng bucket theo cơ chế danh sách liên kết.

Trong các phiên bản OpenJDK hiện đại, khi một bucket có quá nhiều entry, cấu trúc của bucket có thể được chuyển từ danh sách liên kết sang cây đỏ-đen để cải thiện hiệu năng tra cứu.

Mô hình khái quát:

HashMap

|

v

Mảng bucket

|

+-- Bucket 0 -> Entry A -> Entry B

|

+-- Bucket 1 -> Entry C

|

+-- Bucket 2 -> null

|

+-- Bucket 3 -> Entry D -> Entry E

Các entry nằm cùng bucket khi kết quả xác định vị trí bucket của chúng giống nhau. Điều này không nhất thiết có nghĩa là các key bằng nhau.

2.3.3. Cơ chế thêm dữ liệu bằng put()

Khi thực hiện:

map.put("name", "Duy");

HashMap thực hiện các bước khái quát sau:

Bước 1: Tính hash của key

HashMap sử dụng hashCode() của key và một phép biến đổi hash để phân phối giá trị hash tốt hơn.

Trong triển khai OpenJDK hiện đại, có thể hình dung phép biến đổi này như sau:

int hash = key.hashCode();

int spreadHash = hash ^ (hash >>> 16);

Đây là minh họa cho cơ chế triển khai thông dụng, không phải quy tắc API bắt buộc cho mọi implementation.

Bước 2: Xác định bucket

Với bảng có độ dài là lũy thừa của 2, OpenJDK thường sử dụng phép toán tương đương:

int index = (table.length - 1) & hash;

Mục đích là xác định vị trí lưu trữ của entry trong mảng bucket.

Bước 3: Kiểm tra key

Nếu bucket chưa có entry, HashMap thêm entry mới.

Nếu bucket đã có entry, HashMap kiểm tra hash và tính bằng nhau của key. Khi key mới bằng key cũ theo quy tắc so sánh, value của entry cũ được cập nhật.

Bước 4: Xử lý va chạm

Va chạm xảy ra khi nhiều key cùng được ánh xạ đến một bucket.

Ví dụ:

Bucket 0 -> Entry A -> Entry B

HashMap cần so sánh các key trong bucket để xác định có key tương đương hay không.

Bước 5: Mở rộng bảng khi cần

HashMap có thể mở rộng bảng khi số lượng entry vượt ngưỡng dựa trên capacity và load factor.

2.3.4. Capacity và Load Factor

Hai khái niệm quan trọng:

- **Capacity:** số bucket trong bảng.
- **Load factor:** hệ số dùng để xác định ngưỡng mở rộng bảng.

Với load factor mặc định thông dụng là 0.75, ngưỡng mở rộng được tính gần đúng bằng:

Threshold = Capacity × Load Factor

Ví dụ:

Capacity = 16

Load Factor = 0.75

Threshold = 16 × 0.75 = 12

Khi số entry vượt ngưỡng, HashMap thường mở rộng bảng. Việc mở rộng có thể làm tăng chi phí tại thời điểm đó vì các entry phải được phân phối lại vào bảng mới.

Không nên nhầm capacity với số lượng phần tử hiện đang được lưu trữ.

2.3.5. Xử lý va chạm bằng danh sách liên kết và cây đỏ-đen

Trong trường hợp nhiều key cùng rơi vào một bucket, HashMap cần xử lý va chạm.

- **Danh sách liên kết:** các entry trong bucket được liên kết với nhau.
- **Cây đỏ-đen:** trong một số điều kiện triển khai, bucket có thể chuyển sang dạng cây để cải thiện hiệu năng khi số entry quá lớn.

Trong OpenJDK hiện đại, các ngưỡng thường gặp là 8 entry để xem xét treeify và capacity của bảng ít nhất 64 để treeify thay vì tiếp tục resize. Đây là chi tiết triển khai, không nên coi là cam kết bất biến của API.

Độ phức tạp tra cứu thường là O(1) trung bình khi phân phối hash tốt. Trường hợp xấu có thể đạt O(n) với bucket dạng danh sách; cây có thể cải thiện thành O(log n) trong điều kiện phù hợp.

2.3.6. Độ phức tạp của HashMap

Thao tác	Độ phức tạp trung bình	Trường hợp xấu
put(key, value)	O(1)	Có thể tăng do va chạm hoặc resize
get(key)	O(1)	O(n) trong trường hợp bất lợi
remove(key)	O(1)	O(n) trong trường hợp bất lợi
containsKey(key)	O(1)	O(n) trong trường hợp bất lợi

Các độ phức tạp trên là mô hình phân tích phổ biến; hiệu năng thực tế phụ thuộc chất lượng hash, số lượng phần tử và cách triển khai.

2.3.7. Đặc điểm và lưu ý

- HashMap cho phép một key null và nhiều value null.
- HashMap không bảo đảm thứ tự duyệt phần tử.
- HashMap không an toàn khi nhiều luồng cùng sửa đổi mà không có cơ chế đồng bộ.
- Không nên thay đổi những thuộc tính tham gia vào equals() hoặc hashCode() của key trong khi key đang được lưu trong Map.

Ví dụ thực tế: ánh xạ mã nhân viên đến thông tin nhân viên, mã sản phẩm đến giá hoặc user ID đến thông tin tài khoản.

2.4. HashSet

2.4.1. Khái niệm

HashSet là một cấu trúc dữ liệu triển khai interface Set, được dùng để lưu các phần tử duy nhất theo quy tắc xác định tính bằng nhau của đối tượng.

Ví dụ:

```
import java.util.HashSet;

import java.util.Set;


public class Main {

    public static void main(String[] args) {

        Set<String> names = new HashSet<>();


        names.add("Duy");

        names.add("An");

        names.add("Duy");


        System.out.println(names);

        System.out.println(names.size());

    }

}
```

Kết quả:

```
[Duy, An]

2
```

Thứ tự in thực tế có thể khác nhau vì HashSet không bảo đảm thứ tự phần tử.

2.4.2. Cấu trúc bên trong

HashSet được triển khai dựa trên HashMap. Mỗi phần tử của HashSet được lưu làm key trong HashMap; các value sử dụng một đối tượng đánh dấu nội bộ chung.

Mô hình:

```
HashSet

  |
```

v

HashMap

|

+-- Key: "Duy" -> PRESENT

+-- Key: "An"   -> PRESENT

+-- Key: "Minh" -> PRESENT

Do đó, HashSet tận dụng cơ chế hash, xử lý va chạm và so sánh key của HashMap.

2.4.3. Cơ chế thêm phần tử

Khi gọi:

set.add("Duy");

HashSet sử dụng cơ chế của HashMap để xác định vị trí lưu trữ và kiểm tra key.

- Nếu chưa có phần tử tương đương, phần tử được thêm.
- Nếu đã có phần tử tương đương, tập hợp không tạo thêm một phần tử mới.

Giá trị trả về của add() là:

- true: tập hợp đã thay đổi vì phần tử được thêm.
- false: phần tử tương đương đã tồn tại.

Ví dụ:

System.out.println(set.add("Duy")); // true nếu chưa có

System.out.println(set.add("Duy")); // false nếu đã có

2.4.4. Độ phức tạp

Thao tác	Độ phức tạp trung bình
add(element)	O(1)
contains(element)	O(1)
remove(element)	O(1)
Duyệt toàn bộ tập hợp	Phụ thuộc số phần tử và capacity

Trong trường hợp hash phân phối kém, các thao tác có thể chậm hơn đáng kể.

2.4.5. Khi nào sử dụng HashSet?

HashSet phù hợp khi cần:

- Loại bỏ phần tử trùng lặp.
- Kiểm tra nhanh một phần tử có tồn tại hay không.
- Quản lý danh sách ID duy nhất.
- Theo dõi các phần tử đã được xử lý.

Nếu cần duy trì thứ tự chèn, có thể cân nhắc LinkedHashSet. Nếu cần duy trì thứ tự sắp xếp, có thể cân nhắc TreeSet.

2.5. So sánh ArrayList, HashMap và HashSet

Tiêu chí	ArrayList	HashMap	HashSet
Mục đích	Danh sách phần tử	Ánh xạ key-value	Tập hợp phần tử duy nhất
Cấu trúc chính	Mảng động	Mảng bucket	Dựa trên HashMap
Cho phép trùng lặp	Có	Key không trùng theo quy tắc bằng nhau; value có thể trùng	Không
Truy cập	Theo index	Theo key	Kiểm tra sự tồn tại
Thứ tự	Duy trì thứ tự phần tử	Không bảo đảm	Không bảo đảm
Truy cập phổ biến	O(1) theo index	O(1) trung bình	O(1) trung bình
An toàn đa luồng mặc định	Không	Không	Không

Kết luận: nên lựa chọn cấu trúc dữ liệu dựa trên thao tác chính của chương trình. ArrayList thích hợp cho danh sách cần truy cập theo vị trí, HashMap thích hợp cho tra cứu theo key và HashSet thích hợp cho việc quản lý các phần tử duy nhất.

III. HashCode và Equals Contract

3.1. Khái niệm equals()

equals() là phương thức được định nghĩa trong class Object, dùng để xác định hai đối tượng có được coi là bằng nhau theo ý nghĩa logic hay không.

Nếu không được override, Object.equals() so sánh tham chiếu, tương tự việc kiểm tra hai biến có trỏ đến cùng một đối tượng hay không.

Ví dụ:

String a = new String("Duy");

String b = new String("Duy");

System.out.println(a == b); // false

System.out.println(a.equals(b)); // true

Hai biến a và b tham chiếu đến hai đối tượng khác nhau nhưng nội dung String bằng nhau.

Toán tử == với đối tượng thông thường kiểm tra hai tham chiếu có trỏ đến cùng đối tượng hay không; equals() có thể được override để so sánh theo nội dung.

3.2. Khái niệm hashCode()

hashCode() trả về một giá trị kiểu int, được sử dụng bởi các cấu trúc dữ liệu dựa trên hash, chẳng hạn HashMap và HashSet.

Ví dụ:

String name = "Duy";

System.out.println(name.hashCode());

Giá trị hash không phải ID duy nhất của đối tượng. Hai đối tượng khác nhau có thể có cùng hashCode, hiện tượng này gọi là va chạm hash.

3.3. Hợp đồng giữa equals() và hashCode()

Đây là một trong những nguyên tắc quan trọng nhất khi sử dụng HashMap và HashSet.

Hợp đồng quy định:

Quy tắc 1: Nếu hai đối tượng bằng nhau theo equals(), chúng phải có cùng hashCode().

a.equals(b) == true

  => a.hashCode() == b.hashCode()

Quy tắc 2: Nếu hai đối tượng có cùng hashCode(), chúng không nhất thiết phải bằng nhau.

a.hashCode() == b.hashCode()

  !=> a.equals(b)

Quy tắc 3: Kết quả equals() cần có tính nhất quán nếu thông tin dùng để so sánh không thay đổi.

Ngoài ra, equals() cần thỏa mãn tính phản xạ, đối xứng, bắc cầu và phải trả về false khi so sánh với null.

3.4. Vì sao HashMap và HashSet cần cả hai phương thức?

Giả sử HashSet đang chứa một đối tượng nhân viên:

Employee e1 = new Employee(1, "Duy");

Sau đó chương trình tạo một đối tượng mới:

Employee e2 = new Employee(1, "An");

Theo yêu cầu nghiệp vụ, hai nhân viên có cùng id được coi là bằng nhau.

Nếu chỉ override equals() mà không override hashCode(), hai đối tượng bằng nhau có thể tạo ra hash khác nhau. HashSet có thể tìm kiếm hoặc thêm chúng ở các bucket khác nhau, dẫn đến việc không phát hiện phần tử trùng như mong đợi.

Nói cách khác, equals() quyết định hai đối tượng có tương đương về mặt logic hay không; hashCode() giúp cấu trúc hash xác định khu vực cần tìm kiếm.

3.5. Ví dụ triển khai Employee đúng cách

```
import java.util.Objects;


public class Employee {
    private final int id;
    private final String name;


    public Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }


    public int getId() {
        return id;
    }


    public String getName() {
```

```
        return name;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }

        if (!(obj instanceof Employee other)) {
            return false;
        }

        return id == other.id;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

Ở ví dụ này, id là thuộc tính quyết định hai Employee có bằng nhau hay không. Vì vậy, cả equals() và hashCode() đều dựa trên id.

Đoạn instanceof Employee other sử dụng cú pháp pattern matching được hỗ trợ trong các phiên bản Java hiện đại. Với Java cũ hơn, có thể viết kiểm tra kiểu rồi ép kiểu riêng.

Kiểm thử:

```
import java.util.HashSet;

import java.util.Set;
```

```
public class Main {
    public static void main(String[] args) {
        Employee e1 = new Employee(1, "Duy");
        Employee e2 = new Employee(1, "An");

        Set<Employee> employees = new HashSet<>();

        System.out.println(e1.equals(e2)); // true

        employees.add(e1);
        employees.add(e2);

        System.out.println(employees.size()); // 1
        System.out.println(employees.contains(e2)); // true
    }
}
```

Kết quả mong đợi:

true

1

true

Mặc dù tên khác nhau, hai đối tượng được coi là bằng nhau vì cùng có id = 1.

3.6. Lỗi thường gặp: thay đổi key sau khi đưa vào HashMap

Xét một class Employee sử dụng id để tính hashCode và so sánh equals. Nếu id thay đổi sau khi đối tượng được thêm vào HashMap, bucket cũ có thể không còn phù hợp với hash mới.

Khi gọi get() bằng key đó, HashMap có thể không tìm thấy entry như mong đợi.

Vì vậy:

- Nên sử dụng key bất biến khi có thể.
- Không thay đổi các trường tham gia vào equals() và hashCode() trong thời gian đối tượng đang được dùng làm key.

- Khi thiết kế entity, cần cân nhắc kỹ cách xác định tính bằng nhau, đặc biệt với ID được tạo sau khi lưu database.

IV. Concurrent Collections

4.1. Khái niệm

Concurrent Collections là các cấu trúc dữ liệu được thiết kế để hỗ trợ nhiều luồng truy cập đồng thời với các cơ chế phối hợp và bảo vệ dữ liệu phù hợp.

Các collection thông thường như HashMap, HashSet và ArrayList không tự động an toàn khi nhiều luồng cùng sửa đổi.

Ví dụ, nếu hai luồng cùng tăng một biến đếm:

count++;

Đây không phải một thao tác nguyên tử. Hai luồng có thể cùng đọc một giá trị cũ rồi ghi đè kết quả của nhau, gây mất cập nhật.

Concurrent Collections giúp xử lý các tình huống tương tự theo cơ chế phù hợp với từng cấu trúc dữ liệu.

4.2. ConcurrentHashMap

4.2.1. Khái niệm

ConcurrentHashMap là một implementation của ConcurrentMap, cho phép nhiều luồng đọc và cập nhật dữ liệu đồng thời với các cơ chế đồng bộ nội bộ.

Ví dụ:

```
import java.util.concurrent.ConcurrentHashMap;

import java.util.concurrent.ConcurrentMap;


public class Main {

    public static void main(String[] args) {

        ConcurrentMap<String, Integer> counts =

                new ConcurrentHashMap<>();


        counts.put("Java", 1);

        counts.put("Spring", 2);


        counts.merge("Java", 1, Integer::sum);
```

```
        System.out.println(counts);
    }
}
```

Giá trị của key Java sẽ là 2.

4.2.2. Cơ chế hoạt động

Trong các phiên bản OpenJDK hiện đại, ConcurrentHashMap sử dụng các thao tác nguyên tử, CAS và đồng bộ cục bộ khi cần thiết để phối hợp các luồng.

CAS (Compare-And-Set/Compare-And-Swap) là cơ chế cập nhật có điều kiện: chỉ cập nhật nếu giá trị hiện tại vẫn bằng giá trị mong đợi.

Khác với cách khóa toàn bộ Map cho mọi thao tác, ConcurrentHashMap cố gắng hạn chế phạm vi tranh chấp, giúp nhiều thao tác không xung đột có thể tiến hành đồng thời.

Cơ chế cụ thể thay đổi theo phiên bản JDK; không nên mô tả ConcurrentHashMap hiện đại đơn giản là luôn sử dụng một khóa cho mỗi segment như các phiên bản cũ.

4.2.3. Đặc điểm

- Hỗ trợ nhiều luồng đọc và cập nhật.
- Không cho phép key null hoặc value null.
- Không bảo đảm thứ tự duyệt phần tử.
- Các thao tác đơn lẻ như put, get, remove có cơ chế thread-safe.
- Các phương thức như merge() hữu ích khi cập nhật một giá trị theo thao tác được cung cấp.

Lưu ý: thread-safe không đồng nghĩa với việc mọi chuỗi thao tác nhiều bước tự động trở thành nguyên tử.

Ví dụ, đoạn sau có thể gặp race condition:

```
if (!map.containsKey("Java")) {
    map.put("Java", 1);
}
```

Hai luồng có thể cùng kiểm tra thấy key chưa tồn tại trước khi một trong hai thực hiện put().

Nếu muốn thêm khi chưa có, nên sử dụng:

```
map.putIfAbsent("Java", 1);
```

Nếu cần tăng bộ đếm an toàn:

map.merge("Java", 1, Integer::sum);

4.2.4. Ví dụ đếm số lần xuất hiện trong nhiều luồng

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;

public class CounterDemo {
    public static void main(String[] args)
            throws InterruptedException {

        ConcurrentMap<String, Integer> counts =
                new ConcurrentHashMap<>();

        Runnable task = () -> {
            for (int i = 0; i < 1000; i++) {
                counts.merge("Java", 1, Integer::sum);
            }
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(counts.get("Java"));
```

```
    }
}
```

Kết quả mong đợi:

2000

join() giúp luồng chính chờ hai luồng hoàn thành trước khi đọc kết quả.

4.2.5. Khi nào sử dụng ConcurrentHashMap?

- Bộ đếm theo từng loại sự kiện.
- Cache dùng chung giữa nhiều luồng.
- Quản lý trạng thái tác vụ.
- Lưu dữ liệu chia sẻ trong server đa luồng.

Nếu chỉ có một luồng truy cập, HashMap thường đơn giản hơn. Nếu nhiều luồng cần truy cập đồng thời, ConcurrentHashMap là một lựa chọn phù hợp.

4.3. BlockingQueue

4.3.1. Khái niệm

BlockingQueue là một interface trong java.util.concurrent, đại diện cho hàng đợi hỗ trợ các thao tác chờ khi cần thiết.

Hai tình huống chính:

- Hàng đợi đầy: thao tác đưa phần tử vào có thể chờ đến khi có chỗ trống.
- Hàng đợi rỗng: thao tác lấy phần tử có thể chờ đến khi có dữ liệu.

BlockingQueue thường được sử dụng trong mô hình Producer–Consumer.

4.3.2. Mô hình Producer–Consumer

Producer tạo dữ liệu và đưa vào hàng đợi. Consumer lấy dữ liệu từ hàng đợi để xử lý.

```
Producer
  |
  | put(task)
  v
+-----------------+
|  BlockingQueue  |
|  Task 1         |
```

| Task 2 |

| Task 3 |

+----------+

|

| take()

v

Consumer

BlockingQueue đảm nhiệm việc phối hợp giữa các luồng, giảm nhu cầu tự viết cơ chế chờ và đánh thức.

4.3.3. Các implementation phổ biến

Implementation	Đặc điểm
ArrayBlockingQueue	Hàng đợi có sức chứa cố định, dựa trên mảng
LinkedBlockingQueue	Hàng đợi liên kết, có thể giới hạn sức chứa
PriorityBlockingQueue	Lấy phần tử theo thứ tự ưu tiên, không giới hạn dung lượng theo cách thông thường
SynchronousQueue	Không lưu trữ phần tử; mỗi lần đưa vào cần được nhận tương ứng

Việc chọn implementation phụ thuộc vào yêu cầu về sức chứa, thứ tự xử lý và cách phối hợp giữa các luồng.

4.3.4. Các phương thức quan trọng

Phương thức	Khi queue đầy	Khi queue rỗng
put(e)	Chờ	Không áp dụng
take()	Không áp dụng	Chờ
offer(e)	Trả về false	Không áp dụng
poll()	Không áp dụng	Trả về null
offer(e, timeout, unit)	Chờ trong thời gian giới hạn	Không áp dụng
poll(timeout, unit)	Không áp dụng	Chờ trong thời gian giới hạn

put() và take() có thể ném InterruptedException, vì vậy cần xử lý việc ngắt luồng đúng cách.

4.3.5. Ví dụ Producer–Consumer

```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public class ProducerConsumerDemo {
    public static void main(String[] args) {
        BlockingQueue<Integer> queue =
                new ArrayBlockingQueue<>(3);

        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    queue.put(i);
                    System.out.println("Produced: " + i);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    int value = queue.take();
                    System.out.println("Consumed: " + value);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
```

```
        });


        producer.start();
        consumer.start();
    }
}
```

Thứ tự in có thể thay đổi tùy lịch thực thi của hệ điều hành. Tuy nhiên, mỗi giá trị được đưa vào sẽ được lấy ra đúng một lần trong ví dụ này.

Khi bị ngắt, luồng khôi phục trạng thái interrupt bằng Thread.currentThread().interrupt() để thông tin ngắt không bị mất.

4.3.6. Ứng dụng thực tế

BlockingQueue có thể được sử dụng trong:

- Hệ thống xử lý đơn hàng.
- Hàng đợi công việc nền.
- Xử lý tác vụ gửi email hoặc thông báo.
- Pipeline xử lý dữ liệu.
- Điều phối công việc giữa các worker.

Ví dụ, trong hệ thống đặt hàng, một luồng tiếp nhận đơn có thể đưa tác vụ vào hàng đợi, trong khi các worker lấy từng tác vụ để xử lý thanh toán hoặc gửi thông báo.

V. Design Patterns

5.1. Tổng quan

Design Pattern là các mẫu thiết kế đã được đúc kết để giải quyết những vấn đề thường gặp trong thiết kế phần mềm.

Design Pattern không phải là đoạn code bắt buộc phải sao chép nguyên vẹn. Đây là cách tổ chức các class và đối tượng nhằm tăng khả năng tái sử dụng, giảm phụ thuộc và hỗ trợ bảo trì.

Bốn mẫu thiết kế cần tìm hiểu trong tuần này:

Pattern	Nhóm	Mục đích
Singleton	Creational	Giới hạn việc khởi tạo và cung cấp một instance dùng chung theo thiết kế
Factory	Creational	Tách logic tạo đối tượng khỏi nơi sử dụng

Observer	Behavioral	Thông báo cho các đối tượng đăng ký khi trạng thái hoặc sự kiện thay đổi
Strategy	Behavioral	Cho phép thay đổi thuật toán hoặc hành vi thông qua các implementation khác nhau

5.2. Singleton Pattern

5.2.1. Khái niệm

Singleton là Design Pattern nhằm bảo đảm một class cung cấp một instance duy nhất trong phạm vi được thiết kế, đồng thời cung cấp cách truy cập instance đó.

Các thành phần thường gặp:

- Constructor không cho bên ngoài tùy ý khởi tạo.
- Một instance dùng chung.
- Phương thức truy cập instance.

5.2.2. Ví dụ

```
public class AppConfig {
    private static final AppConfig INSTANCE =
            new AppConfig();

    private AppConfig() {
    }

    public static AppConfig getInstance() {
        return INSTANCE;
    }

    public void printConfig() {
        System.out.println("Application configuration");
    }
}
```

Sử dụng:

```
public class Main {
```

```
    public static void main(String[] args) {
        AppConfig a = AppConfig.getInstance();
        AppConfig b = AppConfig.getInstance();

        System.out.println(a == b); // true
    }
}
```

Vì cả hai biến nhận cùng instance nên phép so sánh == trả về true.

5.2.3. Cơ chế hoạt động

- private constructor ngăn class bên ngoài gọi new AppConfig().
- static final lưu tham chiếu đến instance ở cấp class.
- Phương thức getInstance() cung cấp cách lấy instance dùng chung.

Cách khởi tạo eager như trên được Java hỗ trợ an toàn khi class được khởi tạo. Tuy nhiên, instance được tạo ngay khi class được khởi tạo, kể cả khi chưa có nhu cầu sử dụng.

Một lựa chọn ngắn gọn khác là sử dụng enum:

```
public enum AppConfig {
    INSTANCE;

    public void printConfig() {
        System.out.println("Application configuration");
    }
}
```

Java bảo đảm cách khởi tạo enum an toàn về mặt đồng bộ và enum còn có các cơ chế bảo vệ đặc biệt đối với serialization.

5.2.4. Ưu điểm và hạn chế

Ưu điểm:

- Cung cấp một điểm truy cập dùng chung.
- Hạn chế việc tạo nhiều instance ngoài ý muốn.

- Phù hợp với những đối tượng thực sự cần một instance dùng chung trong phạm vi ứng dụng.

Hạn chế:

- Có thể tạo ra trạng thái toàn cục khó kiểm soát.
- Tăng sự phụ thuộc giữa các class.
- Có thể gây khó khăn khi viết unit test nếu đối tượng không thể thay thế.
- Không tự động bảo đảm mọi dữ liệu bên trong instance đều thread-safe.

5.2.5. Khi nào nên sử dụng?

Có thể cân nhắc Singleton cho một số đối tượng cấu hình hoặc dịch vụ dùng chung có vòng đời rõ ràng.

Trong ứng dụng Spring Boot, thường nên ưu tiên Dependency Injection và bean scope phù hợp thay vì tự tạo Singleton thủ công. Bean mặc định của Spring thường là singleton trong phạm vi từng ApplicationContext, nhưng singleton bean không đồng nghĩa với việc các thao tác trên trạng thái mutable của bean tự động thread-safe.

5.3. Factory Pattern

5.3.1. Khái niệm

Factory Pattern giúp tách quá trình tạo đối tượng khỏi phần code sử dụng đối tượng đó.

Thay vì để nhiều nơi trực tiếp khởi tạo các implementation cụ thể bằng new, chương trình có thể gọi một factory để nhận đối tượng phù hợp.

Ví dụ: hệ thống thanh toán có thể hỗ trợ nhiều phương thức như thanh toán tiền mặt, thẻ hoặc ví điện tử.

5.3.2. Ví dụ

Đầu tiên, định nghĩa interface:

```
public interface Payment {
    void pay(double amount);
}
```

Tạo các implementation:

```
public class CashPayment implements Payment {
    @Override
    public void pay(double amount) {
```

```
        System.out.println("Cash payment: " + amount);
    }
}
public class CardPayment implements Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Card payment: " + amount);
    }
}
```

Tiếp theo, tạo factory:

```
public class PaymentFactory {

    public static Payment create(String type) {
        return switch (type.toLowerCase()) {
            case "cash" -> new CashPayment();
            case "card" -> new CardPayment();
            default -> throw new IllegalArgumentException(
                    "Unsupported payment type: " + type);
        };
    }
}
```

Sử dụng:

```
public class Main {
    public static void main(String[] args) {
        Payment payment = PaymentFactory.create("card");
        payment.pay(500000);
    }
}
```

Kết quả:

Card payment: 500000.0

5.3.3. Cách hoạt động

1. Client yêu cầu một loại đối tượng thông qua factory.
2. Factory xác định implementation phù hợp.
3. Factory tạo và trả về đối tượng thông qua interface chung.
4. Client gọi phương thức trên interface mà không cần phụ thuộc trực tiếp vào class cụ thể.

Trong ví dụ này, Main không cần biết cách khởi tạo CardPayment.

5.3.4. Ưu điểm và hạn chế

Ưu điểm:

- Tách biệt logic tạo đối tượng khỏi logic nghiệp vụ.
- Giảm phụ thuộc trực tiếp vào implementation.
- Tập trung xử lý việc lựa chọn loại đối tượng.
- Giúp kiểm soát quá trình tạo đối tượng tốt hơn.

Hạn chế:

- Có thể tạo thêm class và tầng trừu tượng.
- Một factory đơn giản có thể phải sửa mỗi khi thêm loại đối tượng mới.
- Không phải ứng dụng nào cũng cần factory; nếu việc tạo đối tượng rất đơn giản thì sử dụng constructor trực tiếp có thể dễ hiểu hơn.

Lưu ý: ví dụ trên minh họa một Simple Factory. Trong cách phân loại GoF, Factory Method là một pattern riêng, thường cho phép subclass quyết định loại sản phẩm cụ thể được tạo ra.

5.3.5. Ứng dụng trong thực tế

- Tạo đối tượng theo loại thanh toán.
- Tạo các loại thông báo khác nhau.
- Chọn implementation theo cấu hình.
- Tạo các đối tượng kết nối hoặc client theo môi trường.

Trong Spring Boot, có thể kết hợp interface, Dependency Injection và các bean được cấu hình để lựa chọn implementation phù hợp.

5.4. Observer Pattern

5.4.1. Khái niệm

Observer Pattern xác định mối quan hệ một-nhiều giữa các đối tượng. Khi đối tượng được theo dõi phát sinh sự kiện hoặc thay đổi trạng thái, các đối tượng đã đăng ký nhận thông báo sẽ được cập nhật.

Các thành phần chính:

- **Subject/Publisher:** phát sinh sự kiện.
- **Observer/Subscriber:** nhận và xử lý sự kiện.
- **Subscription:** mối quan hệ đăng ký nhận thông báo.

5.4.2. Ví dụ trong hệ thống đặt hàng

Khi khách hàng đặt hàng thành công, hệ thống có thể cần:

- Gửi email xác nhận.
- Gửi thông báo đến ứng dụng.
- Ghi log hoạt động.

Nếu viết trực tiếp toàn bộ logic vào một class OrderService, class này sẽ phụ thuộc vào nhiều thành phần.

Observer Pattern giúp tách các hành động phản ứng với sự kiện đặt hàng thành các observer riêng biệt.

5.4.3. Ví dụ code

Định nghĩa interface:

```
public interface OrderObserver {
    void onOrderCreated(String orderId);
}
```

Observer gửi email:

```
public class EmailObserver implements OrderObserver {
    @Override
    public void onOrderCreated(String orderId) {
        System.out.println(
                "Send email for order: " + orderId);
    }
}
```

Observer ghi log:

```
public class LogObserver implements OrderObserver {
    @Override
    public void onOrderCreated(String orderId) {
        System.out.println(
                "Log order created: " + orderId);
    }
}
```

Subject:

```
import java.util.ArrayList;
import java.util.List;

public class OrderService {
    private final List<OrderObserver> observers =
            new ArrayList<>();

    public void subscribe(OrderObserver observer) {
        observers.add(observer);
    }

    public void createOrder(String orderId) {
        System.out.println("Created order: " + orderId);

        for (OrderObserver observer : observers) {
            observer.onOrderCreated(orderId);
        }
    }
}
```

Sử dụng:

```
public class Main {
```

```
    public static void main(String[] args) {
        OrderService service = new OrderService();

        service.subscribe(new EmailObserver());
        service.subscribe(new LogObserver());

        service.createOrder("ORD001");
    }
}
```

Kết quả:

```
Created order: ORD001
Send email for order: ORD001
Log order created: ORD001
```

5.4.4. Ưu điểm

- Giảm sự phụ thuộc giữa nơi phát sinh sự kiện và nơi xử lý sự kiện.
- Có thể thêm observer mới mà không cần viết lại toàn bộ logic phát sinh sự kiện.
- Cho phép một sự kiện kích hoạt nhiều hành động.
- Phù hợp với hệ thống cần phản ứng với các sự kiện nghiệp vụ.

5.4.5. Hạn chế

- Khó theo dõi luồng thực thi nếu có nhiều observer.
- Một observer có lỗi có thể ảnh hưởng đến quá trình thông báo nếu không được xử lý phù hợp.
- Cần quản lý việc đăng ký và hủy đăng ký để tránh giữ tham chiếu không cần thiết.
- Nếu chạy đồng bộ, observer xử lý lâu có thể làm chậm luồng phát sự kiện.

Ví dụ trên dùng danh sách thường và chỉ phù hợp với tình huống minh họa đơn giản. Nếu nhiều luồng có thể đồng thời đăng ký, hủy đăng ký hoặc phát sự kiện, cần thiết kế cơ chế đồng bộ phù hợp.

5.4.6. Ứng dụng thực tế

- Gửi email khi đơn hàng được tạo.

- Cập nhật giao diện khi dữ liệu thay đổi.
- Ghi log khi có sự kiện nghiệp vụ.
- Phát thông báo khi trạng thái giao dịch thay đổi.
- Xây dựng event-driven architecture ở quy mô phù hợp.

Trong Spring, cơ chế application event cũng cung cấp một cách tổ chức các thành phần phát và nhận sự kiện.

5.5. Strategy Pattern

5.5.1. Khái niệm

Strategy Pattern cho phép xác định một nhóm thuật toán hoặc hành vi thông qua một interface chung, sau đó triển khai từng thuật toán trong các class riêng biệt.

Đối tượng sử dụng có thể lựa chọn chiến lược phù hợp mà không cần chứa toàn bộ logic của các thuật toán.

5.5.2. Ví dụ hệ thống thanh toán

Một đơn hàng có thể được thanh toán bằng nhiều phương thức. Mỗi phương thức có cách xử lý khác nhau, nhưng đều thực hiện cùng một hành động ở mức khái quát là thanh toán.

Định nghĩa interface:

```
public interface PaymentStrategy {
    void pay(double amount);
}
```

Chiến lược thanh toán tiền mặt:

```
public class CashPaymentStrategy
        implements PaymentStrategy {

    @Override
    public void pay(double amount) {
        System.out.println("Pay by cash: " + amount);
    }
}
```

Chiến lược thanh toán bằng thẻ:

```
public class CardPaymentStrategy
```

```
        implements PaymentStrategy {

    @Override
    public void pay(double amount) {
        System.out.println("Pay by card: " + amount);
    }
}
```

Context:

```
public class Order {
    private final PaymentStrategy paymentStrategy;

    public Order(PaymentStrategy paymentStrategy) {
        this.paymentStrategy = paymentStrategy;
    }

    public void checkout(double amount) {
        paymentStrategy.pay(amount);
    }
}
```

Sử dụng:

```
public class Main {
    public static void main(String[] args) {
        Order cashOrder =
                new Order(new CashPaymentStrategy());

        Order cardOrder =
                new Order(new CardPaymentStrategy());

        cashOrder.checkout(100000);
```

```
        cardOrder.checkout(200000);
    }
}
```

Kết quả:

Pay by cash: 100000.0

Pay by card: 200000.0

5.5.3. Cách hoạt động

1. Xác định hành vi chung thông qua interface PaymentStrategy.
2. Tạo từng class triển khai thuật toán cụ thể.
3. Context nhận một strategy thông qua constructor.
4. Khi cần thực hiện hành vi, Context gọi phương thức của strategy được cung cấp.

Điểm quan trọng là Order không cần biết chi tiết triển khai thanh toán tiền mặt hoặc thẻ.

5.5.4. Ưu điểm

- Hạn chế các khối if-else hoặc switch chứa quá nhiều thuật toán.
- Dễ bổ sung chiến lược mới.
- Cho phép thay đổi hành vi mà không phải viết lại toàn bộ class sử dụng.
- Hỗ trợ nguyên tắc Open/Closed và Dependency Inversion khi được áp dụng hợp lý.

5.5.5. Hạn chế

- Có thể làm tăng số lượng class.
- Cần có cơ chế lựa chọn strategy phù hợp.
- Nếu chỉ có một thuật toán đơn giản và ít khả năng thay đổi, việc tạo nhiều strategy có thể không cần thiết.

5.5.6. Ứng dụng thực tế

- Nhiều phương thức thanh toán.
- Nhiều cách tính phí vận chuyển.
- Các thuật toán giảm giá.
- Các phương thức sắp xếp hoặc xử lý dữ liệu.

- Nhiều chính sách tính lương hoặc tính thuế.

Trong dự án Spring Boot, có thể đăng ký nhiều bean triển khai cùng một interface rồi lựa chọn implementation dựa trên loại nghiệp vụ.